

Sequential and Pipelined RISC-V Processor Design

Sanjana Punna Saikiran S Himangi Sahoo
2024102065 2024112007 2024112020

February 2026

Contents

Sequential Implementation	2
Stage 1: Instruction Fetch	3
Program Counter(PC):-	3
Instruction memory(IR):-	4
Stage 2: Instruction Decode	4
Control Unit:-	5
Registers:-	6
Immediate Generator:-	6
Stage 3: Execution	7
ALU Control Unit:-	7
The Arithmetic Logic Unit (ALU):-	8
Stage 4: Memory Access	10
Memory Access Behavior	10
Role of MemRead and MemWrite	11
Stage 5: Write Back	12
Implementation of Fibonacci Sequence Code	12
Pipeline Implementation	15
Why pipelining?	15
Data hazards	19
Forwarding Unit	19
Hazard Control Unit	20
Contributions and Acknowledgment	22
References	22

Sequential Implementation

A sequential processor (also known as a non-pipelined processor) executes every instruction step-by-step in a single processing unit. In this design, the entire instruction completes before the next one begins. Hence, at any moment in time, only one instruction is active inside the CPU.

Our processor only supports the following instructions: **add**, **sub**, **addi**, **and**, **or**, **ld**, **sd** and **beq**.

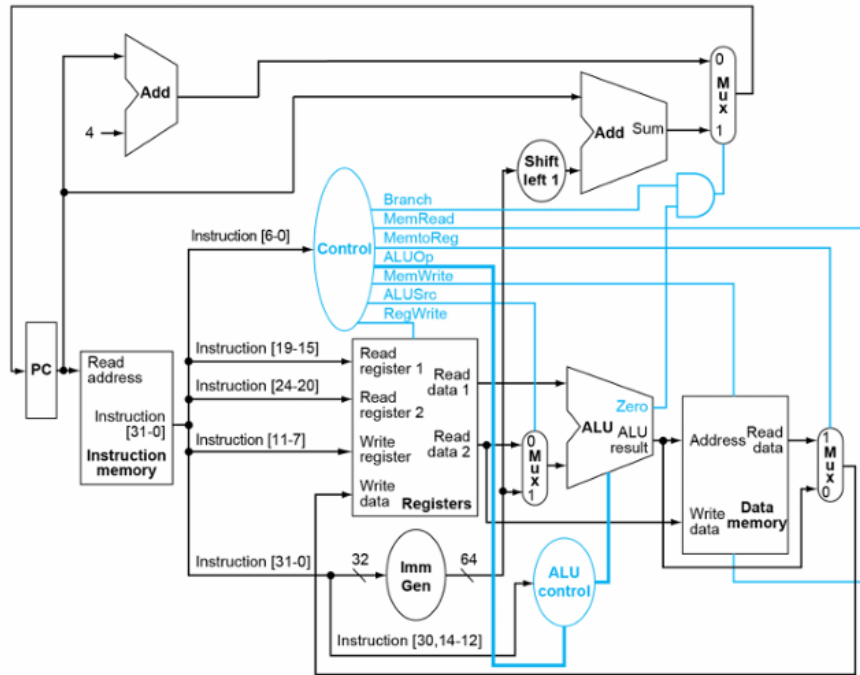


Figure 1: Sequential Datapath

Although the processor is sequential, each instruction still goes through the following five logical stages. The five stages are:

1. Instruction Fetch(IF)
2. Instruction Decode(ID)
3. Execute(EX)
4. Memory Access (MEM)
5. Write Back (WB)

Stage 1: Instruction Fetch

This stage is responsible for retrieving the next instruction from the memory.

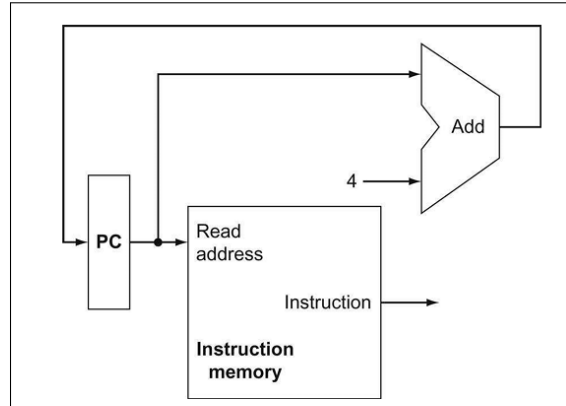


Figure 2: PC and Instruction memory

Program Counter(PC):-

- The Program Counter (PC) is a 64-bit register which holds the address of the instruction to be executed.
- It is basically a pointer which points to the address of the next executable instruction.
- The PC is updated by 4 after each 32-bit (4 bytes) instruction to point to the next instruction. This is done by using an adder.
- The PC is incremented to point to the next instruction.
- A multiplexer (MUX) selects between the next sequential instruction address ($PC + 4$) and the branch target address, depending on the branch control signal.

Verification:

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

PASS at time 10000 | PC = 0000000000000000
PASS at time 20000 | PC = 0000000000000004
PASS at time 30000 | PC = 0000000000000008
PASS at time 40000 | PC = 000000000000000c
PASS at time 50000 | PC = 0000000000000064

ALL TESTS PASSED
pc_tb.v:74: $finish called at 50000 (1ps)

```

Figure 3: Sample Testbench output of Program Counter (PC)

Instruction memory(IR):-

- The address of the next instruction is passed to the Instruction memory.
- It fetches the instructions from the memory and sends the 32-bit instruction to the next stage .i.e, the Instruction Decode Stage.

Verification:

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

PASS at addr 0 | Instruction = 00500113
PASS at addr 4 | Instruction = 00a00193
PASS at addr 8 | Instruction = 003100b3
PASS at addr 12 | Instruction = 40310133
PASS at addr 16 | Instruction = 0031f233
PASS at addr 20 | Instruction = 0041f2b3
PASS at addr 24 | Instruction = 00416333
PASS at addr 28 | Instruction = 003163b3
PASS at addr 32 | Instruction = 0012b023
PASS at addr 36 | Instruction = 0002b503
PASS at addr 40 | Instruction = 0062bc23
PASS at addr 44 | Instruction = 0182b583
PASS at addr 48 | Instruction = 00520463
PASS at addr 52 | Instruction = 00000063
PASS at addr 56 | Instruction = 00a086b3

ALL INSTRUCTION MEMORY TESTS PASSED
instruction_mem_tb.v:56: $finish called at 15000 (1ps)

```

Figure 4: Sample Testbench output of Instruction Memory

Stage 2: Instruction Decode

This stage decodes the fetched instruction to make the processor identify the required operands for execution, by extracting the opcode, register addresses, immediate values etc.

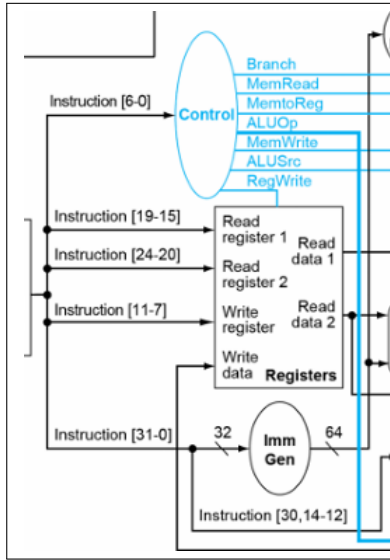


Figure 5: Instruction Decode

Control Unit:-

- The control unit decodes the instruction to determine its meaning - it receives the Opcode [6:0] and generates the corresponding control signals.
- The Opcodes are as follows :-

Instruction	Opcode
R-type	0110011
Load	0000011
Store	0100011
Branch	1100011

- Identifies the required registers and addressing modes.
- Selects the ALU operation or control signals needed for execution.

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

-----Control Unit Tests-----
PASS opcode=0110011
PASS opcode=0000011
PASS opcode=0100011
PASS opcode=1100011
control_unit_tb.v:66: $finish called at 4000 (1ps)

```

Figure 6: Sample Testbench output of Control Unit

Registers:-

- Bits [19:15] and [24:20] are used to read values from the Register File.
- Bits [11:7] are used to store result in the Register File after the operation is executed.

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

PASS at time 11000 | Read1: 0000000000000000 Read2: 0000000000000000
PASS at time 22000 | Read1: aaaabbbbccccddd Read2: 0000000000000000
PASS at time 33000 | Read1: 123456789abcdef0 Read2: aaaabbbbccccddd
PASS at time 44000 | Read1: 0000000000000000 Read2: aaaabbbbccccddd

ALL TESTS PASSED
reg_file_tb.v:133: $finish called at 44000 (1ps)

```

Figure 7: Sample Testbench output of Register

Immediate Generator:-

- It extracts the immediate values for load, store, I-type and B-type instructions.
- it converts the immediate values from 32-bit to 64-bit - it just extends the last bit till 64-bits.

	31	12	11					5	4			0
R	n/a											
I	SE<inst[31]						inst[31:20]					
S	SE<inst[31]						inst[31:25]				inst[11:7]	
B	SE<inst[31]						inst[7 30:25]				[11:8]	
U	inst[31:12]						0	0	0	0	0	0
J	SE<inst[31] inst[19:12] inst[20 30:25] inst[24:21]											

Figure 8: 32-bit Immediate value bits for different types of instruction

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```
---- I-TYPE TEST (addi x2, x0, -5) ----  
Immediate = ffffffffffffffb  
---- I-TYPE TEST (ld x1, 16(x0)) ----  
Immediate = 000000000000010  
---- S-TYPE TEST (sd x6, 24(x5)) ----  
Immediate = 000000000000018  
---- B-TYPE TEST (beq x1, x2, 8) ----  
Immediate = 000000000000008  
ImmGen_tb.v:41: $finish called at 40000 (1ps)
```

Figure 9: Sample Testbench output of Immediate Generator

Stage 3: Execution

This stage fetches the data from the registers and executes the operation decoded by the Instruction Decode Stage.

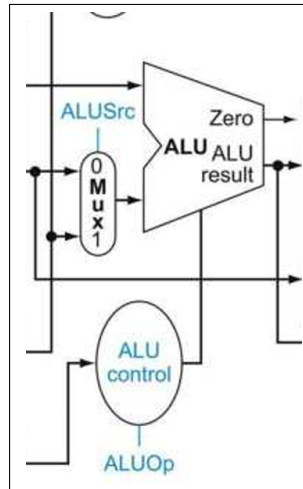


Figure 10: ALU Control and ALU

ALU Control Unit:-

- It fetches the ALUOp code from Control Unit to determine the type of instruction to be operated i.e. S-type, I-type, B-type etc.
- Based on instruction code [31:0], it determines which operation to perform i.e. Add, Subtract, AND, OR, by decoding the funct7 and funct3 field.

- Generates a 4-bit instruction which corresponds to which operation to execute.
- The 4-bit instructions with its corresponding function is as follows :-

ALU control lines	Function
0000	AND
0001	OR
0010	Add
0110	Subtract

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
ld	00	load doubleword	XXXXXXX	XXX	add	0010
sd	00	store doubleword	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Figure 11: ALUOp, ALU control lines and its corresponding functions

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

----- ALU CONTROL TEST -----
ALUOp=00 (load/store) -> ALUControl = 0010 (Expected 0010)
ALUOp=01 (beq) -> ALUControl = 0110 (Expected 0110)
ADD -> ALUControl = 0010 (Expected 0010)
SUB -> ALUControl = 0110 (Expected 0110)
AND -> ALUControl = 0000 (Expected 0000)
OR -> ALUControl = 0001 (Expected 0001)
----- TEST COMPLETE -----
AluCtrl_tb.v:52: $finish called at 60000 (1ps)

```

Figure 12: Sample Testbench output of ALU Control Unit

The Arithmetic Logic Unit (ALU):-

- Receives the 4-bit instruction from ALU control Unit.
- Executes operations on the data fetched from registers.
- Operations may include arithmetic (add, subtract), logical (AND, OR), or address calculation (base + offset).

Stage 4: Memory Access

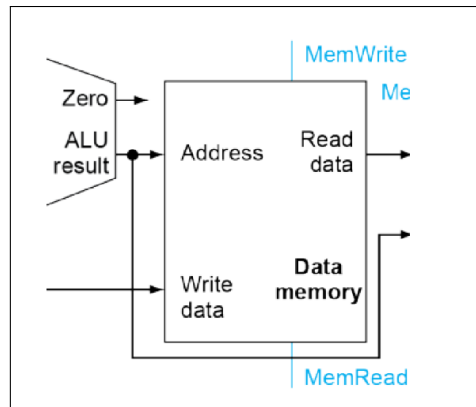


Figure 16: Memory access

Memory Access Behavior

Load Instructions (LD)

- The effective memory address is computed as the *ALU result*.
- The processor reads 64 bits from Data Memory at the computed address.
- The data becomes available at the beginning of the next clock cycle.
- This value is written back into the destination register during the Write-Back stage.

Store Instructions (SD)

- The effective memory address is computed as the *ALU result*.
- The processor writes 64 bits from a source register into Data Memory at this address.
- No value is written back into the register file.
- The next instruction proceeds without any additional write-back activity.

All Other Instructions

- The Memory Access stage performs no operation.
- The ALU output is forwarded directly to the Write-Back multiplexer.

Role of MemRead and MemWrite

These control signals originate from the Control Unit based on the instruction's opcode:

- **MemRead** = 1: The instruction requires reading from Data Memory (e.g., `ld`).
- **MemWrite** = 1: The instruction requires writing to Data Memory (e.g., `sd`).

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
<code>ld</code>	1	1	1	1	0	0	0	0
<code>sd</code>	1	X	0	0	1	0	0	0
<code>beq</code>	0	X	0	0	0	1	0	1

Figure 17: opcodes of the Instructions

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

-----Data Memory Tests-----
PASS address=000000000 write_data=deadbeefcafeba MemRead=0 MemWrite=1 read_data=0000000000000000
PASS address=000000000 write_data=0000000000000000 MemRead=1 MemWrite=0 read_data=deadbeefcafeba
PASS address=000000100 write_data=0000000000000000 MemRead=1 MemWrite=0 read_data=0000000000000000
PASS address=000000100 write_data=1234567890abcdef MemRead=0 MemWrite=1 read_data=0000000000000000
PASS address=000000000 write_data=0000000000000000 MemRead=1 MemWrite=0 read_data=deadbeefcafeba
PASS address=000000100 write_data=0000000000000000 MemRead=1 MemWrite=0 read_data=1234567890abcdef
PASS address=000000000 write_data=ffffffffffffff MemRead=1 MemWrite=0 read_data=deadbeefcafeba
All tests passed!
data_mem_tb.v:88: $finish called at 72000 (1ps)

```

Figure 18: Sample Testbench output of Data Memory module



Figure 19: GTKwave of Data Memory module

Stage 5: Write Back

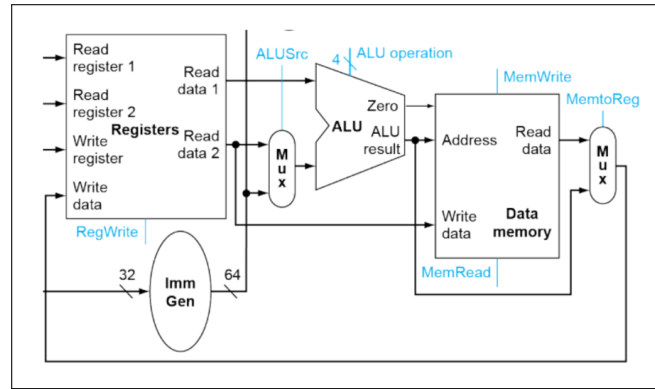


Figure 20: Write back into the register file

- The final output of an instruction is written to the Register File.
 - For ALU instructions, the value produced by the ALU is written to the destination register.
 - For load instructions, the value retrieved from Data Memory is written back instead.
- A multiplexer at the end of the datapath selects whether the Write-Back stage receives the ALU result or the memory output.
- The **RegWrite** signal generated by the Control Unit enables the register file to store the selected value.

Implementation of Fibonacci Sequence Code

We tested our code with a simple iterative version of the first ten elements of the Fibonacci series. The assembly code for implementing this is as shown below.

```

addi x1, x0, 10      /* Calculate 10 Fibonacci numbers*/
addi x2, x0, 0        /* Address for storing results*/
addi x3, x0, 0        /* Fib(0) = 0*/
addi x4, x0, 1        /* Fib(1) = 1*/
sd x3, 0(x2)         /* Store Fib(0)*/
addi x2, x2, 8        /* Increment address (changed from 8)*/
sd x4, 0(x2)         /* Store Fib(1)*/
addi x2, x2, 8        /* Increment address (changed from 8)*/
addi x5, x0, 2        /* i = 2*/
loop:
    beq x5, x1, done
  
```

```

        add x6, x3, x4
        sd x6, 0(x2)
        addi x2, x2, 8
        addi x5, x5, 1
        add x3, x0, x4
        add x4, x0, x6
        beq x0, x0, loop
done:
        add x2, x0, x0          /*iterating address for loading*/
        ld x10, 0(x2)          /*put the series onto the registers (x10–x19)*/
        ld x11, 8(x2)
        ld x12, 16(x2)
        ld x13, 24(x2)
        ld x14, 32(x2)
        ld x15, 40(x2)
        ld x16, 48(x2)
        ld x17, 56(x2)
        ld x18, 64(x2)
        ld x19, 72(x2)

```

At the end of the code we are expected to see registers 10-19 be filled with the first 10 values of the Fibonacci series, which is seen in the output file as follows:

```

0000000000000000
0000000000000000a
0000000000000000
000000000000000015
000000000000000022
0000000000000000a
000000000000000022
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
00000000000000001
00000000000000001
00000000000000002
00000000000000003
00000000000000005
00000000000000008
0000000000000000d
000000000000000015
000000000000000022
0000000000000000
0000000000000000
0000000000000000

```

```

0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000

```

86

The waveform of different control signals, instruction and program counter are plotted to show an instant where the code takes a branch and executes the same set of instructions again is shown below:

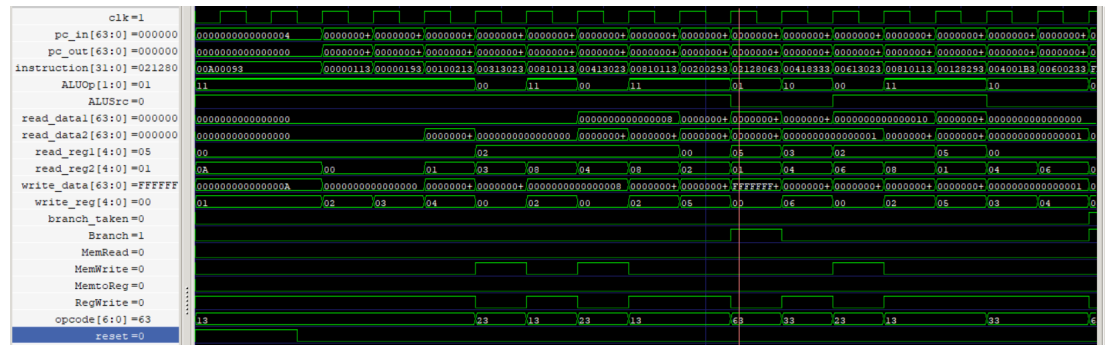


Figure 21: Waveform showing an instant of execution of the fibonacci program.

This concludes our processor's design and verification of the same.

Pipeline Implementation

A pipeline processor executes instructions by dividing the execution process into multiple stages, allowing multiple instructions to be executed simultaneously in different stages, unlike sequential design in which the next instruction's execution is not started until the previous one has been completed.

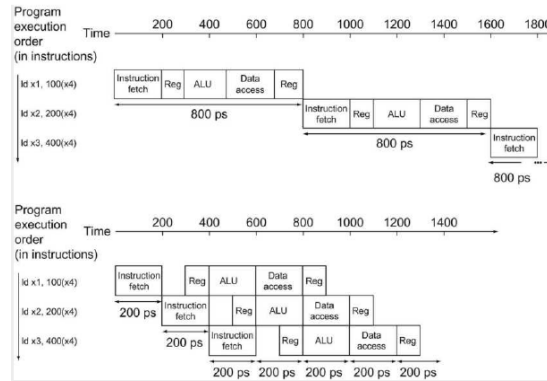


Figure 22: Sequential vs Pipeline Implementation

Here, while one instruction moves from stage 1 to stage 2, the next instruction starts its execution at stage 1, and so on.

Why pipelining?

A pipeline processor generally increases throughput but affects latency, which is a good tradeoff. As we divide the architecture into 5 stages, thus in pipelining, 5 instructions could be executed simultaneously at different stages, thus increasing throughput. However, Latency is compromised in pipelining. From the above picture, 'Reg' takes 100ps whereas 'Instruction Fetch' takes 200ps. In pipelining, when 'Reg' and 'Instruction Fetch' is executing together, it will take the larger time i.e. 200ps. But still, pipelining is preferred over sequential for its higher throughput.

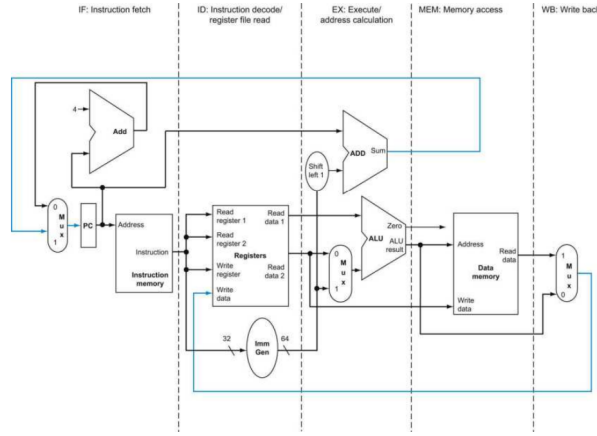


Figure 23: Division of sequential implementation into 5 stages for pipelining it

We start off the pipelined implementation by observing five independent operations that could theoretically be done independently and simultaneously on our processor. These different stages are taken from the von-Neumann standard for computer architecture. These are :

- **Instruction Fetch:** Instruction is Fetched from the instruction memory based on the address pointed to by the current program counter. The next program counter is also calculated in this stage.
- **Instruction decode:** The fetched instruction is then sent to the register file to fetch the appropriate register's data, and the control signals on how different components are to behave are set in this stage.
- **Execute:** The necessary operands along with the necessary opcode as obtained from the control signals and the ALU control unit is obtained and the operation as described by the instruction is performed.
- **Memory:** Sometimes the Result of the ALU might need to be written onto the memory for future use, or data might need to be retrieved from the memory and stored in the register for future operations. According to the instruction type, this stage is used for loading and storing from the memory or is skipped.
- **Write Back:** The data that may be fetched from the memory need to be written on to the general purpose register for future use. This stage ensures that either the values computed from the ALU or the value fetched from the memory are written to the right register based on the control signals for that instruction.

Pipelining Registers: To make sure that each of these stages can execute multiple instructions simultaneously and independently we have registers at the end of each stage called pipelining registers. The pipelining registers must ensure that any information that is required by an instruction at the future stage is available in the registers prior. The registers at the end of each stage are called pipelining registers and are labeled appropriately as follows:

1. **IF/ID Pipeline Registers:-** These are registers placed to separate the Instruction Fetch and instruction decode stages. The IF/ID registers have the following information:-
 - (a) **Instruction:** The instruction fetched from the instruction memory is stored in this registers. The size of this is **64 bits**.
 - (b) **Program counter:** The address of the program counter for the instruction being executed is stored in this register. The size of this register is **64 bits**.
2. **ID/EX Pipeline Registers:-** These are registers placed to separate the Instruction Decode and Execution stages. The ID/EX registers have the following information:-
 - (a) **Control Signals:** It has the control signals generated by the control unit. All are **1-bit** signal. The control signals are *reg_write_en*, *out_mux*, *mem_read_out_mux*, *mem_to_reg_out_mux*, *mem_write_out_mux*.
 - (b) **Data from Register:** It has **64-bits** data from the Register File (*read_data1*, *read_data2*). It also stores **5-bits** *rd*, *rs1*, *rs2* for forward checks.
 - (c) **Immediate value:** It gets the **64-bit** immediate value (*imm*) from the Immediate Generator.
 - (d) **Opcodes:** It gets the **4-bits** Opcodes from ALU control unit.
3. **EX/MEM Pipeline Register:-** These are registers placed to separate the Execute and Memory stages. The EX/MEM registers have the following information:-
 - (a) **Control Signals:** The control signals for the memory stage and the write back stage are present in these registers. They include the following control signals associated with the instruction :*Branch*, *MemRead*, *MemWrite*, *MemtoReg* and *Regwrite*. Each control signal takes 1 bit so the size of this register is **5 bits**.
 - (b) **ALU results:** The result computed by the ALU along with the zero flag is stored in this register for future processing - totaling to **65 bits** of information.
 - (c) **Rs2 Data:** In Case of a load instruction we might need the second register's data addressed to by the instruction as it is the data to be written onto the memory. To allow for this we also store this register's data. The size of this is **64 bits**.

- (d) **Destination Register:** The destination register should be used in the write back stage to drive any computed result to the appropriate register. Apart from this it is also used for forwarding checks when any computed result accessed before write back needs to be forwarded. The size of this is **64 bits**.
4. **MEM/WB Pipeline Register:-** These are registers placed to separate the Memory and Write Back stages. The MEM/WB registers have the following information:-
- (a) **Control Signals:** The control signals for the write back stage are present in these registers. They include the following control signals associated with the instruction : *MemtoReg* and *Regwrite*. Each control signal takes 1 bit so the size of this register is **2 bits**.
 - (b) **Read Data:** We might need the data read from the memory at the memory address specified in case of a load operation. Thus, necessitating this register. The size of this register is **64 bits**.
 - (c) **ALU output:** In case that the instruction is not a load type we need to forward our output from the ALU to one of the registers. Hence we might need this output aswell necessitating a need to store these values before the write back stage. The size of this register is **64 bits**.
 - (d) **Destination register:** We need the destination address of the register to which either the ALU output or data needs to be stored hence we need a register to store this destination address. This is also used in forwarding the result of the MEM stage according to the destination address. The size of this register is **5 bits**.

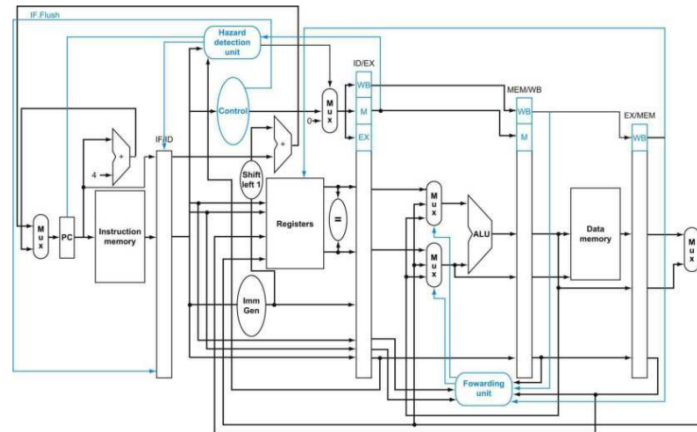


Figure 24: Complete Pipelined processor

Data hazards

In pipelined processors, multiple instructions are executed in overlapping stages. While this improves performance, it can also create situations where an instruction depends on data that another instruction has not yet produced. These kind of situations are called **Data Hazards**. The most common ways to prevent data hazards are forwarding and stalling. We stall when the data to be sent to the register has not yet been produced or computed.

Forwarding Unit

We forward our data when the data that is required is available at the output of EX stage or the MEM stage and give it to a dependent instruction in EX stage. To prevent this hazards we needed to implement 2 muxes to forward the required data from the `EX/MEM.RegisterRd` or `MEM/WB.RegisterRd` to the instruction in EX stage. These are called the Forwarding Multiplexors. This forwarding control will be in the EX stage, because the ALU forwarding multiplexers are found in that stage.

The conditions for detecting hazards, and the control signals to resolve them:

1. **EX Hazard:**

```
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 10
```

```
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 10
```

This case forwards the result from the previous instruction to either input of the ALU. If the previous instruction is going to write to the register file, and the write register number matches the read register number of ALU inputs A or B, provided it is not register 0, then steer the multiplexor to pick the value instead from the pipeline register `EX/MEM`.

2. **MEM Hazard:**

```
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 01
```

```
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0)
```

```

and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 01

```

When checking for hazards coming from the MEM/WB stage, the logic verifies whether the destination register in the MEM/WB pipeline register is different from the source registers `rs1` / `rs2` currently in the EX/MEM stage, and that the MEM/WB stage is actually performing a write. We also add a safeguard to avoid forwarding values into register `x0` since it should never be modified.

Moreover, to correctly deal with situations where two forwarding opportunities exist at the same time, we must ensure that the forwarding path from the EX/MEM stage is not active. This is because the EX/MEM forwarding has a higher precedence, so the MEM/WB forwarding only applies when the EX/MEM condition does not already trigger.

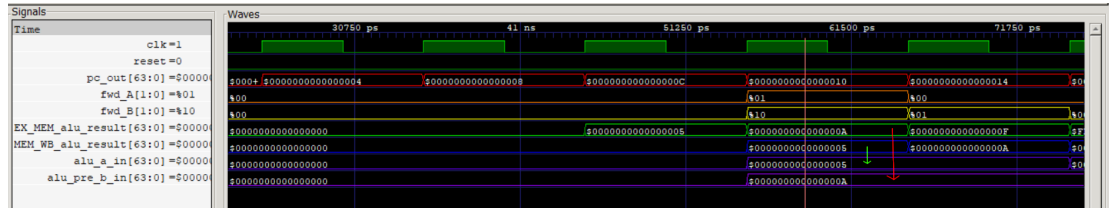


Figure 25: Waveform showing an instant of forwarding. Green arrow shows MEM/WB to EX forwarding and Red arrow shows EX/MEM to EX forwarding.

Hazard Control Unit

The Hazard units take care of the processor stalling when the data from a register that is to be used in an instruction has not yet been computed by a previous instruction issued to the processor. For our small subset of the RV64I instruction set only one such hazard arises.

1. Load-use Hazard:

In this case a load instruction loading some memory content to some register (say `x5`) is immediately followed by any instruction that uses the contents of this register should cause a stall as the data is produced by the lead instruction at the Write back stage which cannot be forwarded to the EX stage on time as by then the instruction would have been on the MEM stage. To make sure that correct data is given to the ex stage we stall the pipeline based on the following condition:

```

if (ID/EX.MemRead
and ((ID/EX.RegisterRd = IF/ID.RegisterRs1)
or (ID/EX.RegisterRd = IF/ID.RegisterRs2))
stall the pipeline

```

If the condition holds, the instruction stalls one clock cycle. After this one-cycle stall, the forwarding logic can handle the dependence and execution proceeds. As an example consider the following:

Example:

```
addi x1 , x0, 1000
sd x1 , 0(x0)
ld x2 , 0(x0)
add x3, x1, x2
add x13, x1, x10
```

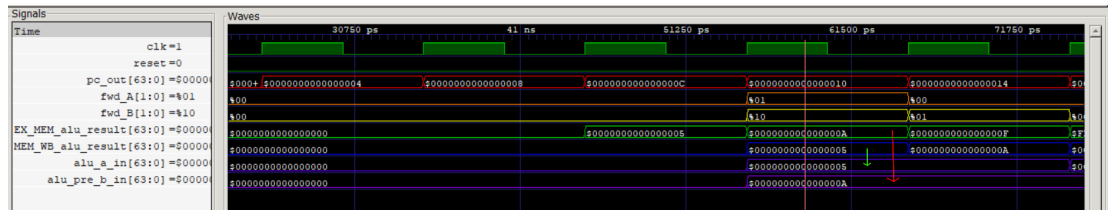


Figure 26: Waveform showing an instant of Load-use hazard stalling. Pink box shows all control signals being set to zero and PC remaining the same, indicating a stall.

2. Control Hazard:

The cases discussed above are often followed by forwarding. Now we will discuss the cases of a flush in case of a branch. We assume that the branch is not taken when we fetch the instruction. If the prediction turns out to be wrong, i.e., the branch is taken, we have to flush the pipeline. In this case, we have to change the PC value to the branch target address and insert `nops` in the pipeline. This is done by flushing the IF/ID and ID/EX register files since they will already have the two wrong instructions fetched after the branch instruction.

Example:

```
sd x6, 24(x5)
ld x11, 24(x5)
beq x4, x5, 8
beq x0, x0, 0
add x13, x1, x10
```

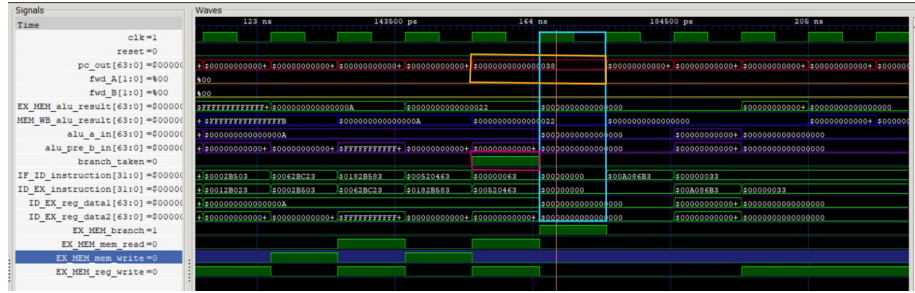


Figure 27: Waveform showing an instant of Branch taken. 1) Orange box shows PC being stalled for 2 clock cycles. 2) Blue box shows the branch not taken and register contents of IF/ID and ID/EX being flushed. 3) Red box shows the branch condition being activated.

Contributions and Acknowledgment

The project was completed collaboratively, with all three members actively involved in brainstorming, implementation, and debugging. While components in the datapath are listed based on primary contributions, the design and decisions were made collectively.

Sanjana worked on the Program Counter (PC), Register file, and Instruction Memory, and her ALU module has been used in the processor from the previous assignment. Himangi implemented the Immediate Generator, ALU Control, and Control Unit. Saikiran worked on Data memory block, Forwarding unit, and Pipeline Register. The final integration of sequential and pipelining implementation has been a group effort of all the members. Test cases for individual blocks, Fibonacci sequence, Data and Control Hazards etc. were designed and evaluated collaboratively.

We sincerely thank Prof. Priyesh Shukla and the Teaching Assistants for their invaluable guidance.

References

- [1] RISC-V Assembler Tool. Available at:
<https://riscvasm.lucasteske.dev/>
- [2] David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.
- [3] RISC-V 32I Quick Reference Card; FM extensions, RARS implementation. CC-BY-SA downtothewires (via YouTube).